Computer Systems Organization

https://nyu-cso.github.io

Jinyang Li

Course information

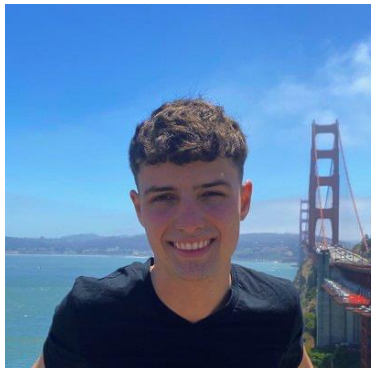
Lecturer: Prof. Jinyang Li



MW 3:30-4:45pm, 60 Fifth Ave 150

Recitation instructor:

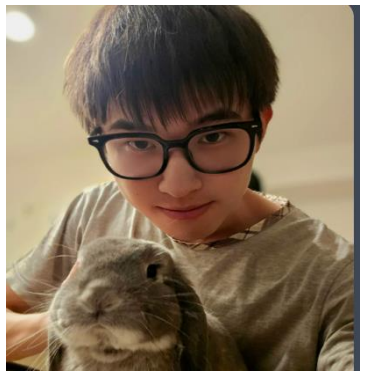
David Pissarra



F 12:30-1:45pm, Silv 206

Course Assistants:

Zhanghan Wang & David Pissarra

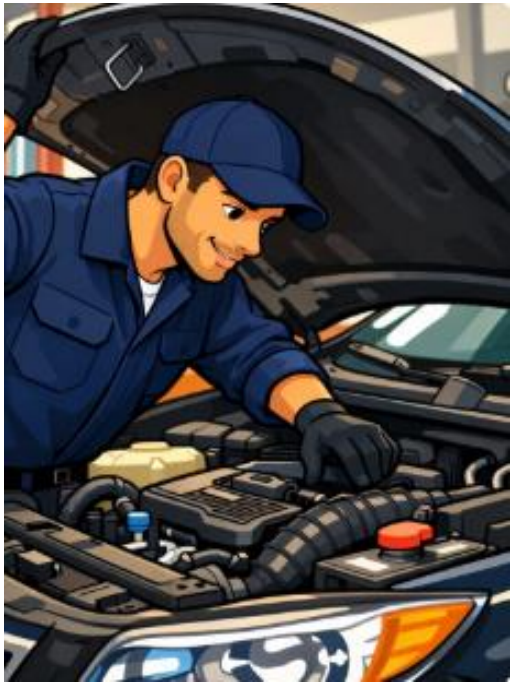


Course Goal

- Beyond learning how to program
  - Learn the gritty internals of how a computer system works



You after CS101, 102

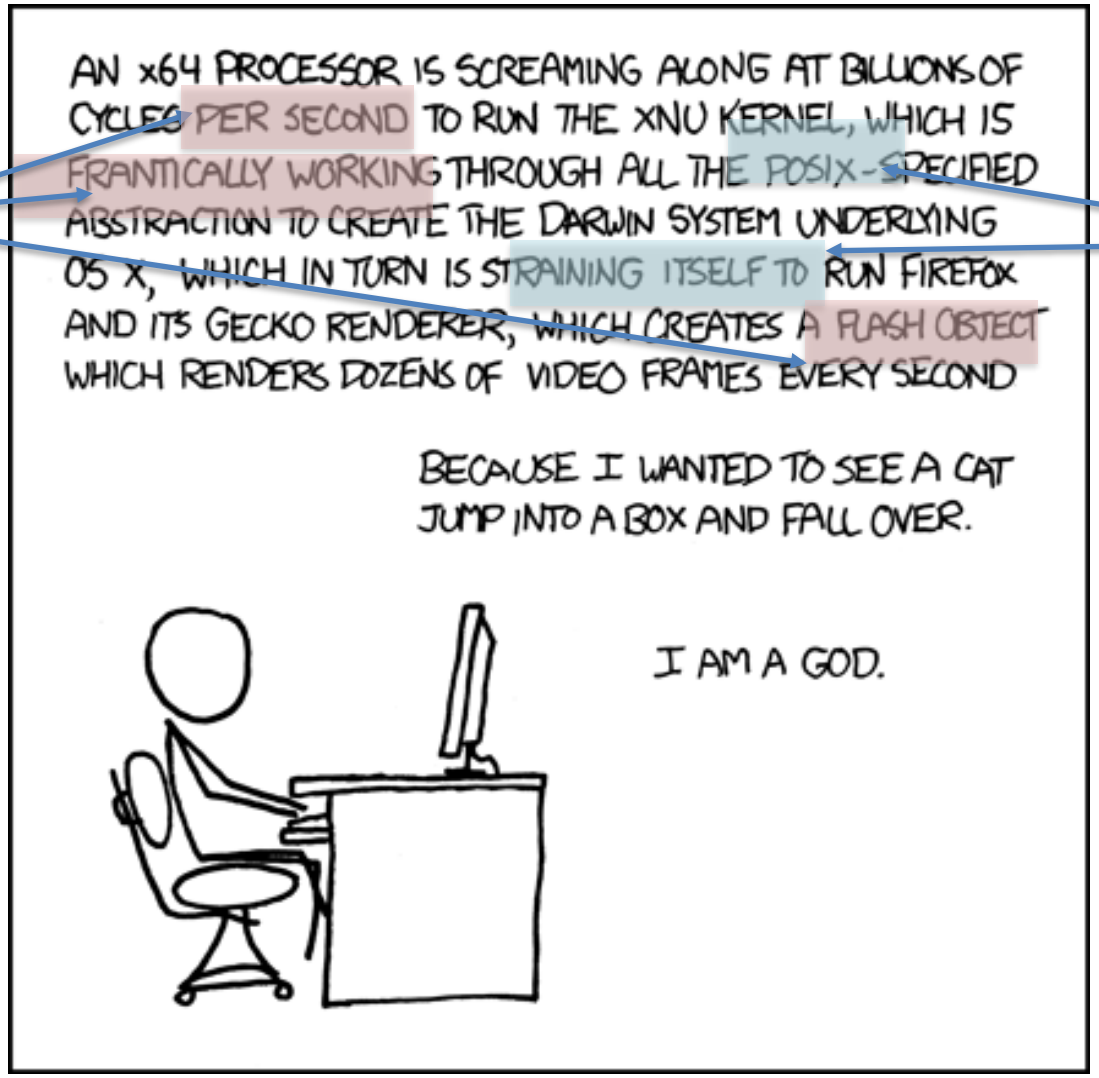


You after CS201, 202

Goal: learn how computer systems work

Covered by CSO

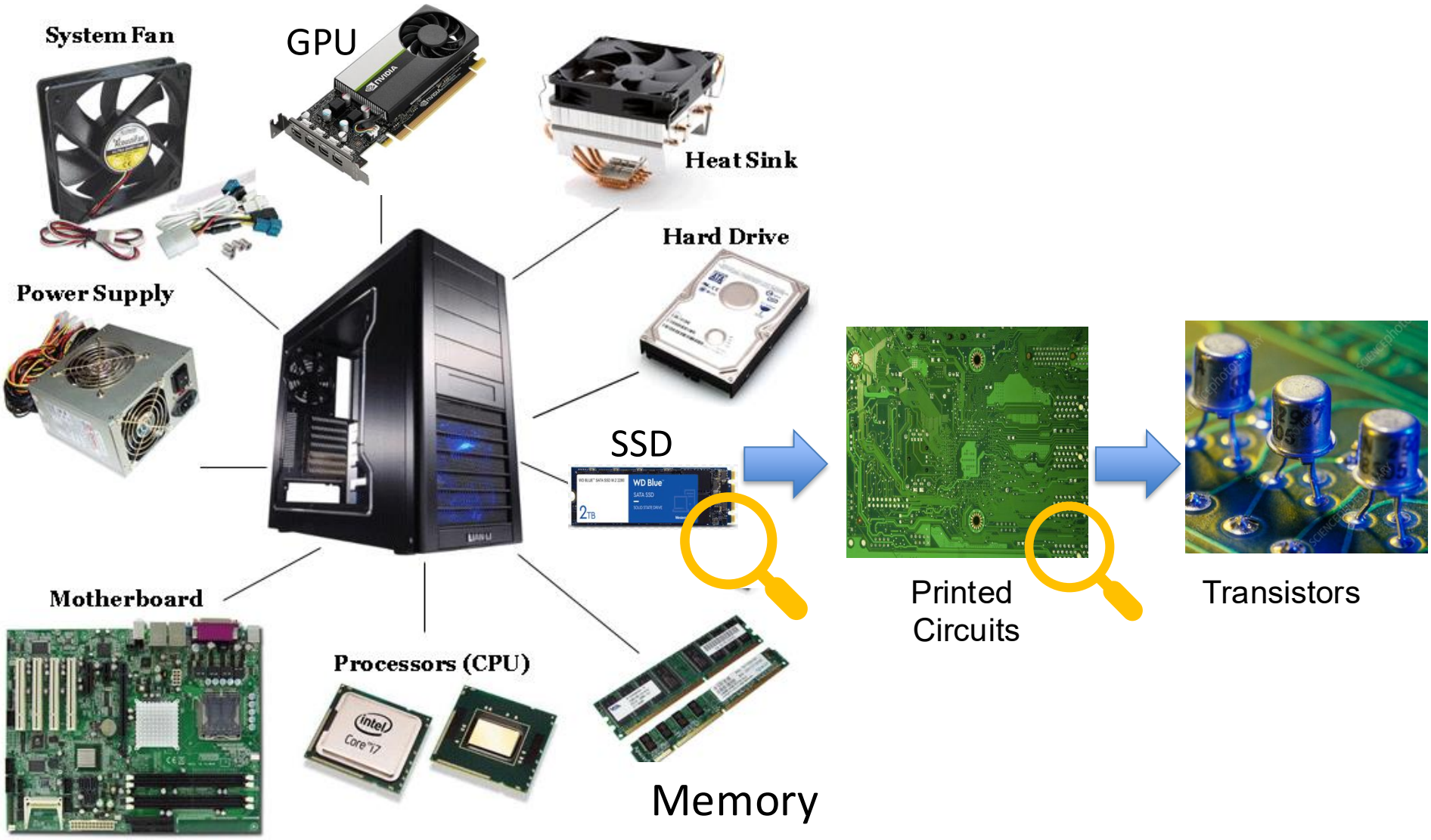
To be covered by OS (202)



Components of a computer



Components of a computer: hardware



Components of a computer: hardware + software



Layered Organization

Software



Hardware



Layered Organization



Software

Hardware



Transistors



Diodes



Resistors

Layered Organization



Software

Hardware

Logical Circuits,
Flip-Flops, Gates









Transistors

Diodes

Resistors

Layered Organization

Software



Hardware

CPU, Memory, Disk







Logical Circuits,
Flip-Flops, Gates









Transistors



Diodes



Resistors

Layered Organization

Software



Hardware

CPU | Memory | I/O

Logical Circuits, Flip-Flops, Gates, …

Transistors, Diodes, Resistors, …

Layered Organization

System Software
(OS, compiler, VM…)

Software



Hardware

CPU | Memory | I/O

Logical Circuits, Flip-Flops, Gates, …

Transistors, Diodes, Resistors, …

Layered Organization



Layered Organization

Users



User Applications



System Software
(OS, compiler, VM…)



Software

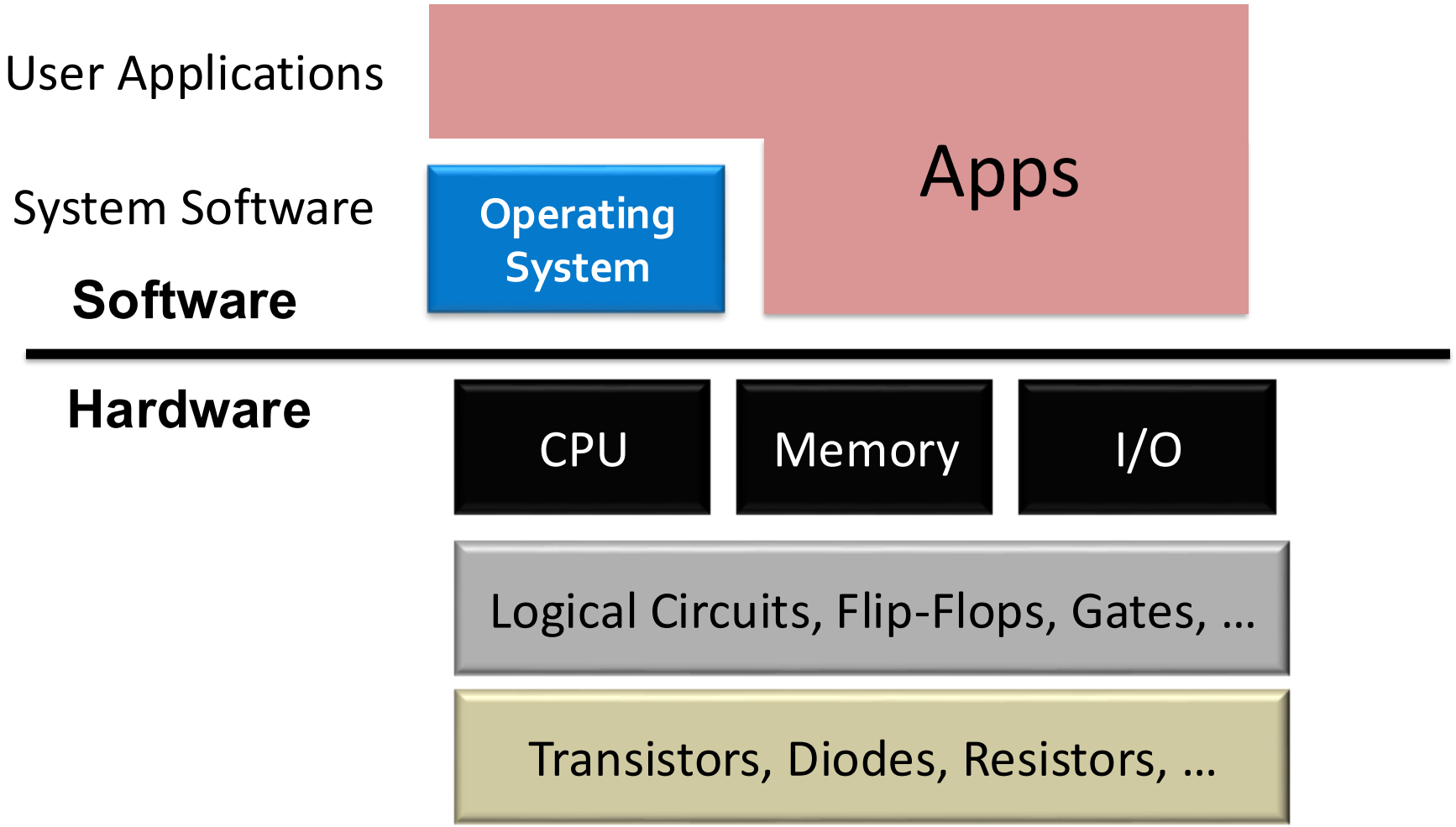
Hardware

CPU | Memory | I/O

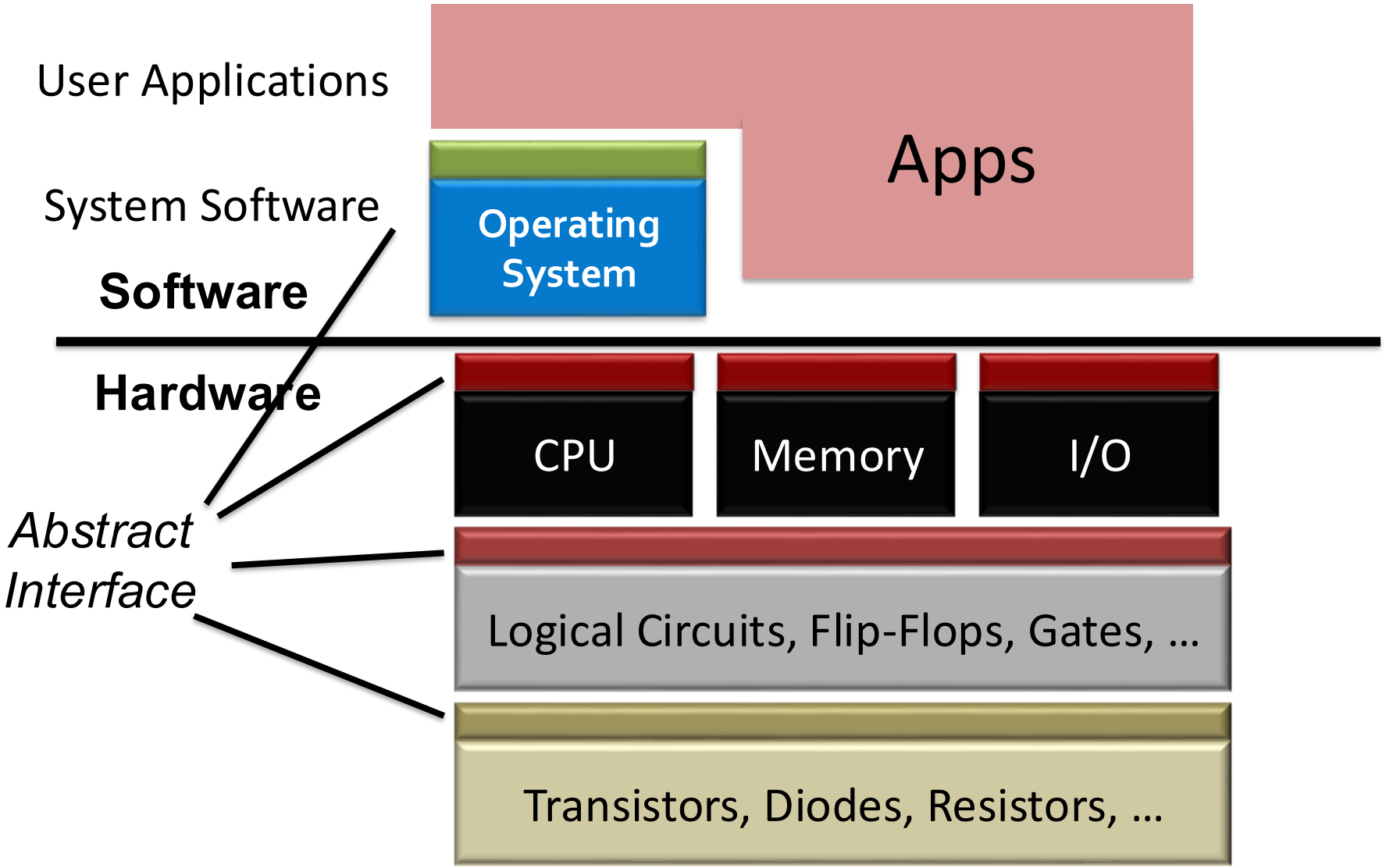
Logical Circuits, Flip-Flops, Gates, …

Transistors, Diodes, Resistors, …

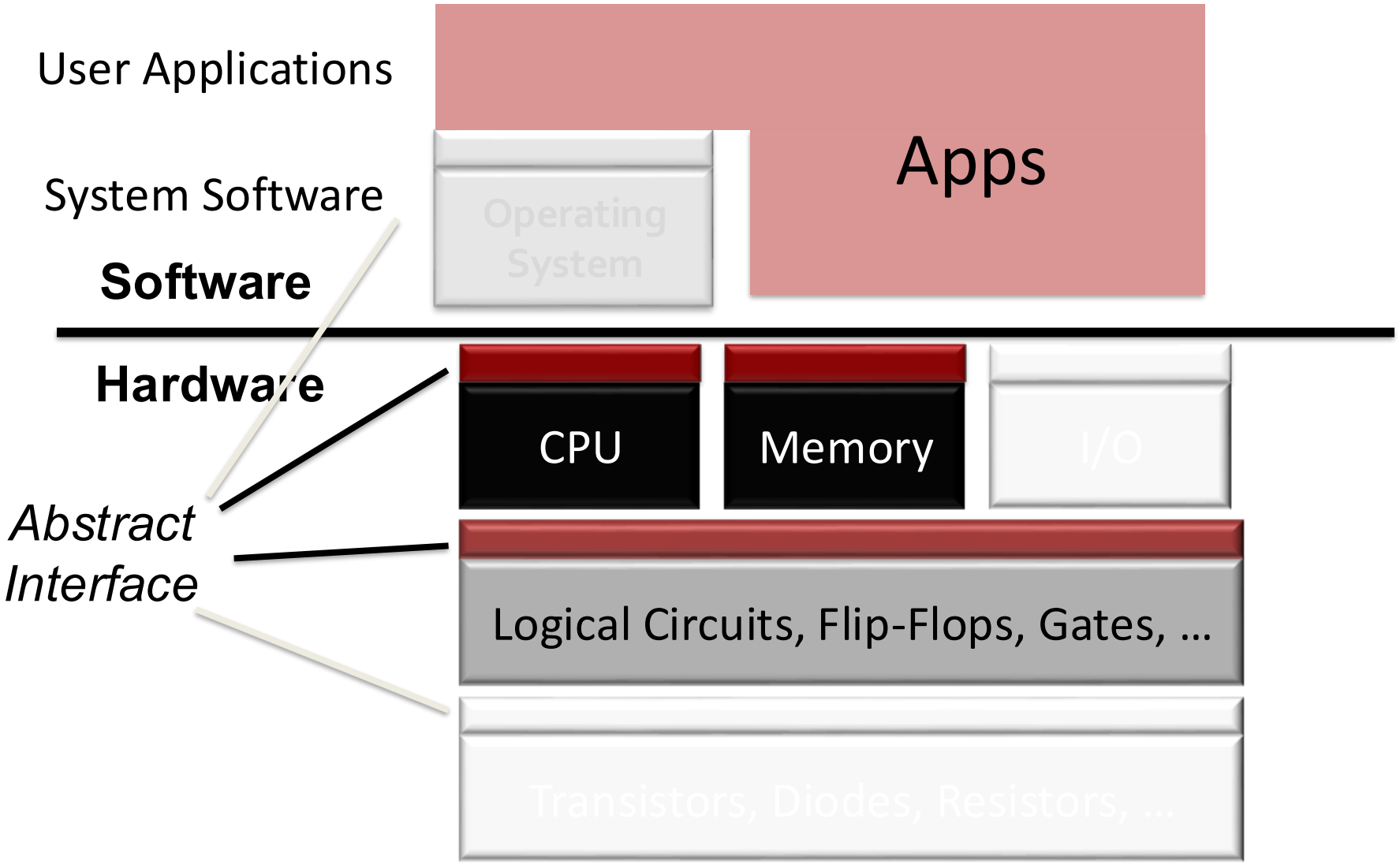
Layered Organization



Abstraction



Scope of this class



Scope of this class

1. How do applications run on a computer?
   - Hardware/software interface
2. How do CPU/memory work?
   - overview of computer architecture

Schedule

https://nyu-cso.github.io

overview
bit, byte and int
float point
[C] basics, bitwise operator, control flow
[C] scopes rules, pointers, arrays
[C] structs, mallocs
[C] large program (linked list)

C Programming

Schedule

https://nyu-cso.github.io

overview
bit, byte and int
float point
[C] basics, bitwise operator, control flow
[C] scopes rules, pointers, arrays
[C] structs, mallocs
[C] large program (linked list)
Machine Prog: ISA, Compile, movq
Machine Prog: Control Code (condition, jump instruction)
Machine Prog: Array allocation and access
Machine Prog: Procedure calls
Machine Prog: Structure, Memory Layout
Machine Prog: Buffer Overflow

C Programming



Assembly (X86)

Schedule

https://nyu-cso.github.io

overview
bit, byte and int
float point
[C] basics, bitwise operator, control flow
[C] scopes rules, pointers, arrays
[C] structs, mallocs
[C] large program (linked list)
Machine Prog: ISA, Compile, movq
Machine Prog: Control Code (condition, jump instruction)
Machine Prog: Array allocation and access
Machine Prog: Procedure calls
Machine Prog: Structure, Memory Layout
Machine Prog: Buffer Overflow
Code optimizations
Dynamic Memory Allocation
Dynamic Memory Allocation continued

C Programming

↓

Assembly (X86)

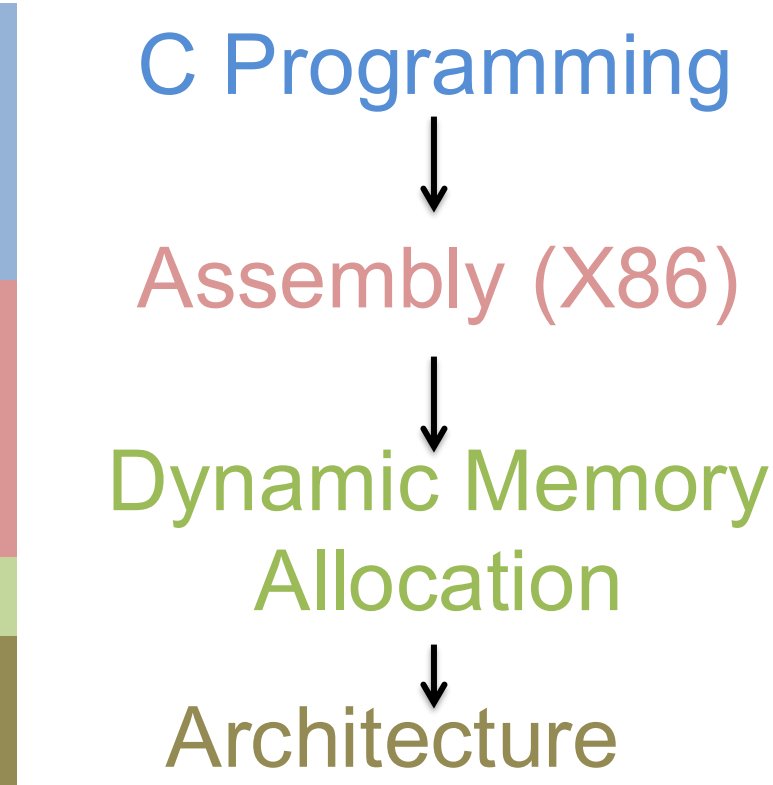
↓

Dynamic Memory Allocation

Schedule

https://nyu-cso.github.io

overview
bit, byte and int
float point
[C] basics, bitwise operator, control flow
[C] scopes rules, pointers, arrays
[C] structs, mallocs
[C] large program (linked list)
Machine Prog: ISA, Compile, movq
Machine Prog: Control Code (condition, jump instruction)
Machine Prog: Array allocation and access
Machine Prog: Procedure calls
Machine Prog: Structure, Memory Layout
Machine Prog: Buffer Overflow
Code optimizations
Dynamic Memory Allocation
Dynamic Memory Allocation continued
Logic Design
Logic Design continued
Sequential implementation
Pipelined implementation



Schedule

https://nyu-cso.github.io

overview
bit, byte and int
float point
[C] basics, bitwise operator, control flow
[C] scopes rules, pointers, arrays
[C] structs, mallocs
[C] large program (linked list)
Machine Prog: ISA, Compile, movq
Machine Prog: Control Code (condition, jump instruction)
Machine Prog: Array allocation and access
Machine Prog: Procedure calls
Machine Prog: Structure, Memory Layout
Machine Prog: Buffer Overflow
Code optimizations
Virtual memory: Address Spaces/ Translation, Goal
Virtual memory: Page table/physcial to virtual
Process
Dynamic Memory Allocation I: malloc, free
Dynamic Memory Allocation II: design allocator
Dynamic Memory Allocation III: futher optimization
Memory, cache
Memory, cache

C Programming

↓

Assembly (X86)

↓

Dynamic Memory Allocation

↓

Architecture

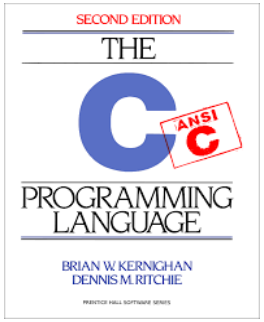
↓

Memory & Cache

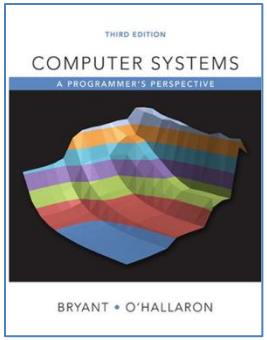
Course logistics

- Website: https://nyu-cso.github.io
  - Syllabus
    - Reading preparation
    - lecture/recitation slides
    - Lab instructions
- Forum: Campuswire
  - Q&A
- NYU Brightspace
  - Gradescope
    - Lab submission, weekly assessments
  - Zoom links, Zoom recordings
  - Use Campuswire instead of Brightspace for Q&A.

Textbooks



The C Programming Language 2nd ed,
Kernighan and Ritchie



Computer Systems -- A programmer's perspective,
3rd ed, Bryant and O'Hallaron.



Computer organization and design (RISC-V edition),
Patterson and Hennessy

Grade Breakdown

- 5 programming labs
  - 1% each
- 4 in-person tests
  - Each test covers a corresponding programming lab plus related concepts.
  - 11% each
- Class participation
  - Recitation participation (5%)
  - Other participation (lecture, online forum): 1%
- In-person final exam (80 minutes)
  - 45%

5 individual programming labs

- Programming environment:
  - (Mostly) your laptop
  - Learn to use:
    - a text editor to write code
    - git for version control
- Submission:
  - Push to github
  - Submit and have it graded via Gradescope
- Late policy:
  - 6 (cumulative) grace days in total over the semester.
  - 3 max. grace days for each lab.

Recitation mini-quiz

- Start next week
- To be done in-class in-person via Gradescope:
  - 15 minutes in the beginning
    - Multiple choice questions and short answers
    - Based on current week's lecture materials
  - Answers discussed immediately during recitation
- Counts towards recitation participation: 5%

To thrive in CSO, you should …

- Before lecture:
  - Read assigned book chapters
- During lecture/recitation:
  - Ask questions
  - Don’t be shy to ask me to repeat.
- Labs
  - Start early
- Getting help:
  - Campuswire
  - Office hours (see post on Campuswire)

Integrity, Collaboration and AI Policy

Almost all the evaluation is based on in-person test/exam.

This means “yes” to:

- All forms of collaboration.
  - E.g. work on programming labs with your buddies
  - Study together etc.
- AI tools (including coding agents) and traditional web search and other online materials
  - Acknowledge your usage for such tools for each lab.
  - You have no AI nor internet access during tests/exams.

Warning: Do not outsource your thinking to AI. Train your brain—or you’ll do badly on the exams.